Security Assessment Report

EtherFi Lend

July-August 2026

Prepared for Ether.Fi

Table of Contents

Project Summary

Project Scope

Project Name	Repository Link	PR Link	Initial Commit Hash	Final Commit Hash	Platform
EtherFi Lend	https://github.com/etherfi-protocol/cash-v3	https://github.com/etherfi-protocol/cash-v3/pull/153	3a401863ef6f4a093a9007c4c849c38d155c7342	5b85936048385aedf6df8931b21d09f4bddd4a44	Solidity
EtherFi Lend Additional Change	https://github.com/etherfi-protocol/cash-v3	https://github.com/etherfi-protocol/cash-v3/pull/261	904ba7a643d131f56b0e5547d1271203c1de5b76	904ba7a643d131f56b0e5547d1271203c1de5b76	Solidity

Project Overview

This document describes the security review of **EtherFi Lend**. The work was undertaken from **July 20th, 2026** to **Aug 4th, 2026**.

The following contract list is included in our scope:

src/modules/lend-gateway/LendGateway.sol
src/modules/lend-gateway/LendCapacityLib.sol
src/modules/cash/CashLendLib.sol
src/modules/cash/CashLensLegacyLib.sol
src/modules/cash/CashLens.sol
src/modules/cash/CashModuleCore.sol
src/modules/cash/CashModuleSetters.sol

src/modules/cash/CashModuleStorageContract.sol
src/modules/cash/CashEventEmitter.sol
src/debt-manager/DebtManagerCore.sol
src/debt-manager/DebtManagerStorageContract.sol
src/hook/EtherFiHook.sol
src/lens/AaveV4Lens.sol
src/libraries/LendSourcingLib.sol
src/libraries/CashVerificationLib.sol
src/modules/ModuleLendGatewaySandwich.sol
src/modules/ModuleCheckBalance.sol
src/modules/etherfi/EtherFiLiquidModule.sol
src/modules/etherfi/EtherFiLiquidModuleWithReferrer.sol
src/modules/etherfi/EtherFiStakeModule.sol
src/modules/etherfi/LiquidUSDLiquifierOP.sol
src/modules/frax/FraxModule.sol
src/modules/hype/BeHYPEStakeModule.sol
src/modules/midas/MidasModule.sol
src/modules/openocean-swap/OpenOceanSwapModule.sol

src/modules/recovery/SafeAssetRecoveryModule.sol

src/enso/EnsoSwapModule.sol

src/across/AcrossSwapModule.sol
src/top-up/TopUpDest.sol
src/safe/RecoveryManager.sol

scripts/lend/CashLendProdConfig.sol

scripts/lend/DeployCashLendProd.s.sol

scripts/lend/VerifyCashLendProd.s.sol

The team performed a manual audit of all the files in scope**.** During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Etherfi's Cash V3 is an on-chain card/spend protocol where users spend from a smart-contract wallet ("EtherFiSafe") in debit mode (spending held collateral) or credit mode (borrowing against it), coordinated by CashModule. Two lending engines coexist during a migration: a legacy DebtManager with its own token whitelists, and a newer LendGateway routing safes into real Aave v4 markets, selected per-safe via usesLendGateway.

Peripheral modules (swaps, bridging/recovery, staking/liquid-asset integrations) sandwich their operations around the gateway — pulling shortfalls from Aave, executing, then best-effort resupplying. CashLens is the read-only quoting layer mirroring execution-side sizing so off-chain systems can pre-check transactions, generally erring conservative versus what execution allows. Withdrawals go through a timelocked pending-request mechanism, with spend/repay flows able to cancel and reclaim reserved funds when needed. Authorization layers owner-quorum multisig, single-admin roles, and a gateway driver allowlist, with Aave's own health-factor checks as the final backstop.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Assets

- Aave v4 Collateral & Debt Positions: Gateway-registered reserves supplied per-safe (e.g. weETH, eBTC, USDC, LiquidUSD) and the borrowed debt against them, spanning both the legacy DebtManager and the new LendGateway engines.
- Safe Loose Balances: Unsupplied ERC-20 balances sitting in an EtherFiSafe awaiting auto-supply, spend, or recovery.
- Card Settlement Capacity: The ability for a `spend()` call (debit or credit) to succeed atomically at swipe time — the single flow the design treats as must-not-fail.
- Withdrawal-Reserved Funds: Balances pulled out of Aave under a pending, timelocked withdrawal request, along with any in-flight bridge/swap order built on top of them (Across, Enso, LiquidModule).
- Owner-Quorum Authorizations: Signed borrow/spend authorizations from Safe owners, and the multisig-gated admin roles (`LEND_GATEWAY_ADMIN_ROLE`, `CASH_MODULE_CONTROLLER_ROLE`) that configure the gateway and CashModule.
- Risk/Observability Data: `CashLens`/`getSafeCashData` output consumed by off-chain risk tooling to assess a safe's real health and borrowing headroom.

Actors

- Safe Owners: Control an EtherFiSafe via owner-quorum signatures; can request lend opt-out, sign borrow authorizations, and reconfigure modules.
- EtherFi Wallet / Card Network: The permissioned caller that triggers `spend()` at the moment of a real-world card swipe.
- Protocol Admins (`LEND_GATEWAY_ADMIN_ROLE`, `CASH_MODULE_CONTROLLER_ROLE`): Configure reserves, spend assets, health-factor floors, and driver allowlists.
- Drivers (`isDriver` role): Purpose-built contracts (DebtManager, TopUpDest, swap/liquifier modules) granted direct withdraw/borrow access into any registered safe's Aave position.
- Aave v4 Hub & Spoke: The external lending venue whose shared Hub-wide liquidity, reserve pause state, and health-factor enforcement EtherFi Lend builds directly on top of.

- Outside/Unrelated Aave Users: Any third party interacting with the same shared Aave Hub/Spoke, whose borrowing or supply activity can incidentally affect EtherFi safes.
- Malicious Actors / Griefers: Attackers who send dust to a targeted safe, park a tiny position on a shared reserve, or time transactions to destroy in-flight operations or brick admin functions, at negligible cost.

Trust Assumptions

- Aave Hub Liquidity Availability: The protocol assumes the shared Aave Hub will generally hold enough idle liquidity for withdrawals, debit spends, and opt-out unwinding to succeed, with no reserve buffer of its own to fall back on.
- Reserve Registry Consistency: The gateway's registered/borrowable/spend-asset sets and the legacy DebtManager's whitelist sets are assumed to stay in sync, though nothing enforces this and they are maintained independently.
- Driver Role Discipline: `isDriver` is trusted to be granted only to scoped, purpose-built contracts, since the role itself imposes no on-chain restriction on withdrawing or borrowing any safe's full position to an arbitrary address.
- Safe Contract Boundaries: EtherFiSafe is assumed to never implement ERC-1271, since doing so would let a safe route around the gateway's accounting entirely via Aave's native signature-based position-manager delegation.
- CashLens as a Faithful Mirror: Off-chain systems trust `CashLens`/`getSafeCashData` to conservatively and accurately reflect what on-chain execution will actually allow, including in over-leveraged or edge-case states.
- One-Time, Order-Sensitive Configuration: Per-safe setup (`setupModule`, engine migration) is assumed to run exactly once and in the audited sequence, with no built-in guard enforcing that assumption at the contract level.

Attack Vectors

- Settlement-Critical Reverts: A capped, frozen, or paused Aave reserve selected during automatic collateral resupply (`_sizeResupply`) has no fallback and no try/catch, so a single unsuppliable reserve can revert an entire credit-mode card swipe even when healthy collateral is available elsewhere.
- Cheap Griefing via Shared State: Because guards like `removeReserve`'s "no open position" check and `migrateToLendGateway`'s per-token supply loop read Aave/Safe-wide state

rather than EtherFi's own accounting, an outsider can permanently brick reserve de-registration or block a safe's migration by parking a trivial position or sending 1 wei of an unregistered token.

- Health-Factor Floor Bypass: Divergent gating between `usesLendGateway` and `isLendActive` lets a safe stuck mid-opt-out (matured timestamp, unwind blocked by open debt) withdraw Aave collateral while completely skipping the configured health-factor floor, and the LiquidUSD repayment path independently assumes debt-for-collateral swaps are always de-risking without verifying it, both allowing positions to land at or below Aave's raw liquidation boundary.
- In-Flight Operation Destruction: Credit-spend resupply sizing cancels a safe's pending withdrawal (and any bridge/swap built on it) whenever a conservative first-pass estimate shows any shortfall, even when the second pass never actually needed the reclaimed funds — silently burning an owner-quorum-signed operation.
- Privileged Bypass of Audited Flows: `setupModule`'s lack of a one-time guard lets a Safe owner replay setup through module removal/re-addition, flipping `usesLendGateway` and resetting spend limits without going through the audited `migrateToLendGateway` path.
- Misclassified Recoverable Assets: `SafeAssetRecoveryModule` checks only the legacy DebtManager's whitelists, so a token registered and actively in use on the gateway (but absent from the legacy lists) can be misclassified as "unsupported" and fully swept out of a safe via `recover()`.
- Deficit and Breach Masking: Both `LendCapacityLib`'s clamp-to-zero capacity/headroom math and `getSafeCashData.maxBorrow`'s clamped sum collapse an over-LTV or already-deficient position to look identical to a healthy one sitting exactly at its limit, breaking the resupply/repay paths meant to recover it and hiding the breach from risk tooling built on the lens.
- Systemic Liquidity Dependency: Shared Aave Hub liquidity and per-reserve pause state gate operations (debit spend, withdrawal, opt-out, even Aave-independent loose-balance settlement) that don't strictly need to touch Aave at all, so unrelated third-party Aave activity can stall EtherFi-specific flows with no fallback buffer.
- Input and Sequencing Gaps: Missing validation on empty/zero-amount withdrawal requests, repayment sizing computed before its own first leg's state change is applied, and unconditional release of reserved withdrawal funds without checking the safe's current headroom needs each create narrow correctness gaps that can produce spurious reverts or misordered fund releases under adversarial timing.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	1	1	1
Low	8	8	3
Informational	11	11	2
Total	20	20	6

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
			Likelihood	

Detailed Findings

ID	Title	Severity	Status
M-01	`_spendGatewayCredit` may revert in certain cases	Medium	Fixed
L-01	`removeReserve` can be DoSed with direct supply	Low	Acknowledged
L-02	`migrateToLendGateway` reverts if any Safe balance of a DebtManager collateral token is not a registered gateway reserve	Low	Acknowledged
L-03	Opted-out Safes with a live Aave position bypass the health-factor floor entirely while still being able to pull collateral	Low	Fixed
L-04	`setupModule` can be replayed to bypass delayed configuration and unilaterally flip the engine flag	Low	Acknowledged
L-05	Migration can create positions below the configured health-factor floor	Low	Acknowledged
L-06	Credit-spend collateral resupply unconditionally destroys the safe's pending withdrawal request (and any in-flight bridge) even when no reserved balance is used or usable	Low	Acknowledged

L-07	SafeAssetRecoveryModule doesn't recognize gateway-registered assets as "supported"	Low	Fixed
L-08	Clamped-to-zero Aave capacity hides existing deficits, breaking credit resupply and repay-triggered withdrawal sizing	Low	Fixed
I-01	EtherFi operations dependent on Aave Hub liquidity	Informational	Acknowledged
I-02	An Aave reserve pause blocks Debit spending even when all required funds are loose in the Safe	Informational	Fixed
I-03	Owner-signed borrow authorizations have no contract domain or expiry	Informational	Acknowledged
I-04	Zero-factor collateral is incorrectly treated as unconditionally withdrawable	Informational	Acknowledged
I-05	Potential to bypass Lend operations and supply and withdraw from Aave directly if safes implement ERC1271	Informational	Acknowledged
I-06	The LiquidUSD repayment path can worsen Aave health while bypassing the configured floor	Informational	Acknowledged
I-07	Repayment sourcing ignores liquidity created by its own first repayment leg	Informational	Fixed
I-08	_requestWithdrawal accepts empty token lists and zero-amount entries	Informational	Fixed

I-09	isDriver role grants unrestricted withdraw/borrow-to-arbitrary-address power over every gateway safe	Informational	Acknowledged
I-10	processWithdrawal doesn't defer to the safe's current headroom needs before releasing reserved funds	Informational	Acknowledged
I-11	getSafeCashData.maxBorrow clamps to debtUsd on an over-LTV position, hiding the breach	Informational	Fixed

Medium Severity Issues

M-01: `_spendGatewayCredit` may revert in certain cases

Severity: Medium	Impact: High	Likelihood: Low
Files: src/modules/cash/CashLendLib.sol	Status: Fixed	

Description:

When a credit spend needs more Aave borrowing power than the safe currently has, `_resupplyForCreditShortfall` tops up collateral by walking the registry (`_sizeResupply`) and supplying loose balances — but its only filter is `gateway.ltv(token) != 0`. It never checks whether the reserve can actually accept a supply right now (frozen, paused, `active`/`halted`, or at its `addCap`), and the execution loop in `_resupplyCollateral` calls `gateway.supply` with no try/catch. So if the safe holds a loose balance of a reserve that is at its supply cap or paused, `_sizeResupply` still selects it, `gateway.supply` reverts, and the entire `CashModule.spend` reverts — breaking card settlement, the one flow the design states "must never fail." This is not a dev-only edge case: the production `LAUNCH_SPEC` sets finite supply caps on every reserve (e.g. weETH at 1,000, eBTC at 200), so a popular collateral hitting its cap is a normal growth state, and small caps are also cheaply grief-able by an outsider filling the remaining headroom (recoverable capital that earns yield while it blocks). Every other supply site in the codebase is deliberately best-effort; this settlement-critical one is the only one that is not.

Recommendations:

Add a view to `LendGateway` that reports whether a reserve can accept a supply *right now* (not frozen/paused, spoke `active` and not `halted`, and `addCap` headroom remaining), and use it in `_sizeResupply` so it skips unsuppliable reserves and moves to the next candidate. Make the `_resupplyCollateral` execution loop resilient too — attempt each supply in `try/catch` and continue — so a reserve that becomes unsuppliable between sizing and execution can't brick the spend. With both in place, a credit spend only fails when there is genuinely no suppliable collateral to cover the shortfall, rather than because a capped or paused reserve was blindly

selected while healthy collateral existed. Add a regression test covering a capped/paused collateral reserve with a healthy alternative, asserting the spend still settles.

Customer Response:

Fixed in PR#241/#242

Fix Review:

Fixed

Low Severity Issues

L-01: `removeReserve` can be DoSed with direct supply

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/modules/lend-gateway/LendGateway.sol	Status: Acknowledged	

Description:

`LendGateway.removeReserve` and `LendGateway.setReserveId` refuse to run while the reserve still
has supplied assets or debt on the Spoke. The check reads a **spoke-wide** total, not EtherFi's own exposure, so any outside address can make it non-zero by parking a tiny position on the Spoke and never closing it. That permanently bricks both functions for the targeted reserve — the admin can no longer delist the asset or re-point it to a corrected reserve. It costs the griefer a few units of dust plus
gas, needs no privileges, and has no clean recovery.

Recommendations:

Stop gating a gateway-owned action on a Spoke-wide total that outsiders can move. Options:

1. Gate on EtherFi's **own** accounted exposure rather than the Spoke aggregate — e.g. track supplied /
borrowed per registered safe inside the gateway (or iterate the gateway's own accounting) and allow de-registration when no EtherFi safe holds a position, ignoring third-party dust.

2. If the Spoke aggregate must be used, add an admin override (a `LEND_GATEWAY_ADMIN_ROLE`-only force path) so a griefer cannot permanently pin a reserve, and document that de-registration ignores non-safe positions.

3. Reconsider `receiveSharesEnabled: true` on EtherFi reserves if a non-safe becoming a Spoke supplier is undesirable for other reasons — though note this only closes Vector B;

Vector A (direct self-supply) remains and is the reason the aggregate-based guard is unsafe in the first place.

Customer Response:

Acknowledged - "*Freezing the reserve is the deprecation path — no new supply or borrows, exits stay open — so delisting is never required and a standing admin force path is not worth its risk*"

Fix Review:

Acknowledged

L-02: `migrateToLendGateway` reverts if any Safe balance of a DebtManager collateral token is not a registered gateway reserve

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/debt-manager/DebtManagerCore.sol	Status: Acknowledged	

Description:

During migration, `migrateToLendGateway` loops through every `DebtManager` collateral token the Safe holds and calls `_gateway.supply` for the raw ERC-20 balance. `LendGateway.supply` explicitly reverts `AssetNotRegistered` for any asset not in the gateway registry.

Because the legacy `DebtManager` collateral list and the gateway registry are allowed to diverge, any Safe holding a legacy collateral token that Aave does not list will cause the migration transaction to permanently revert. Furthermore, an attacker can transfer 1 wei of an unregistered legacy token to a targeted Safe to indefinitely block its migration.

Recommendations:

Check if the token is registered via `_gateway.isRegistered(collateralTokens[i])` before attempting to supply it. If unregistered, skip the supply operation and leave it as a loose balance in the Safe.

Customer Response:

Acknowledged

Fix Review:

Acknowledged

L-03: Opted-out Safes with a live Aave position bypass the health-factor floor entirely while still being able to pull collateral

Severity: Low	Impact: Low	Likelihood: Low
Files: src/modules/ModuleLendGatewaySandwich.sol	Status: Fixed	

Description:

The sandwich gating logic relies asymmetrically on `usesLendGateway` for withdrawals and `isLendActive` for resupply and floor checks. A lend opt-out becomes effective at finalize time. If a user opened debt during the opt-out delay window, the opt-out process permanently blocks unwinding, but `isLendOptedOut` continues to return true.

In this state, any sandwiched module can withdraw supplied collateral out of Aave, keep the output loose, and completely skip the `_ensureGatewayFloor` check. This bypasses the configured protocol-side protections against liquidation, permitting users to park highly levered positions precisely at Aave's raw 1.0 liquidation boundary.

Recommendations:

Gate `_ensureGatewayFloor` (and optionally `_resupplyToGateway`) on `_onGatewayEngine(safe)` rather than `_lendActive(safe)`. The final floor check should run whenever the Safe uses the gateway and collateral was withdrawn, regardless of opt-out status.

Customer Response:

Fixed in commit 6e6202b

Fix Review:

Fixed

L-04: `setupModule` can be replayed to bypass delayed configuration and unilaterally flip the engine flag

Severity: Low	Impact: Low	Likelihood: Low
Files: src/modules/cash/CashModuleCore.sol	Status: Acknowledged	

Description:

`setupModule` in `CashModuleCore` has no one-time initialization guard and is explicitly re-runnable. Safe owners can remove `CashModule` from the local module set and add it again through `configureModules`, causing `setupModule` to execute with arbitrary new setup data.

Replaying setup can immediately reset the spending limits and counters, force the stored mode to `Debit`, and irreversibly set `usesLendGateway` for any debt-free legacy Safe. This allows moving a Safe to the gateway engine by an owner configuration transaction, completely bypassing the audited `migrateToLendGateway` process.

Recommendations:

Track a per-Safe `setupComplete` flag and reject repeated setup. Enforce the intended `CannotRemoveCashModule` guard in `_configureModules`.

Customer Response:

Acknowledged

Fix Review:

Acknowledged

L-05: Migration can create positions below the configured health-factor floor

Severity: Low	Impact: Low	Likelihood: Low
Files: src/debt-manager/DebtManagerCore.sol	Status: Acknowledged	

Description:

Migration validates the re-borrow against `rawWithdrawHeadroom`, which is calculated at Aave's liquidation boundary of `1e18`. It then borrows the migrated debt and marks the Safe as using the gateway without calling `ensureMinHealthFactor`.

The fixed 10-bps migration padding is unrelated to the configured `minHealthFactor`, which may be higher. A migration can leave a Safe near HF `1.001` while the configured protocol floor is `1.2`. This exposes previously healthy users and blocks strictly de-risking module operations.

Recommendations:

After all Aave borrows and before setting `usesLendGateway`, call `_gateway.ensureMinHealthFactor(safe);`. Alternatively, expose migration capacity calculated at the configured floor and gate each re-borrow against it.

Customer Response:

Acknowledged

Fix Review:

Acknowledged

L-06 Credit-spend collateral resupply unconditionally destroys the safe's pending withdrawal request (and any in-flight bridge) even when no reserved balance is used or usable

Severity: **Low**	Impact: **Low**	Likelihood: **Low**
Files: Files	Status: Acknowledged	

Description: `resupplyCollateral` is the recovery path a gateway credit spend takes when borrowing capacity drops. It sizes the resupply in two passes. `cancelOldWithdrawal` is called whenever the first pass leaves any residual shortfallValue, regardless of whether the second pass actually claimed a single wei of reserved balance.

Because shortfallValue is padded up by RESUPPLY_BUFFER_BPS and sizing carry-over is conservative, a residual is essentially systematic. The second pass frequently cannot use anything, but the cancellation is not rolled back. This silently destroys an in-flight AcrossSwapModule / EnsoSwapModule order or EtherFiLiquidModule bridge, consuming an owner-quorum signature.

Recommendations: Make the cancellation conditional on the second pass actually having claimed a reserved balance.

Customer's response: Acknowledged

Fix Review: Acknowledged

L-07 SafeAssetRecoveryModule doesn't recognize gateway-registered assets as "supported"

Severity: **Low**	Impact: **Medium**	Likelihood: **Low**
Files: src/modules/recovery/SafeAssetRecoveryModule.sol	Status: Fixed	

Description: `_assertRecoverable` only checks the legacy DebtManager's collateral/borrow/withdraw-whitelist sets to decide if a token is safe to sweep. It never checks the `LendGateway's` registered/borrowable/spend asset sets, which are maintained independently and can diverge from DebtManager's lists. For a gateway-migrated safe, a token registered on the gateway but absent from DebtManager's legacy lists is misclassified as unsupported and can be fully swept via `recover()`, even while it's meaningfully in use by that safe's Aave V4 position (e.g. loose funds awaiting auto-supply).

Recommendations: Branch `_assertRecoverable` on `cashModule.usesLendGateway(safe)`: for gateway safes, also reject tokens where lendGateway.isRegistered/isBorrowable/isSpendAsset is true; keep the existing `DebtManager` checks for legacy safes. If gateway-safe recovery isn't intended to be supported yet, restrict recover() to legacy safes only until the gateway-aware check is added.

Customer's response: Fixed in commit f6edba2

Fix Review: Fixed - *"The contract issues has been fixed, while the upgrade would happen later as a separate transaction to the main lend deploy script"*

L-08 Clamped-to-zero Aave capacity hides existing deficits, breaking credit resupply and repay-triggered withdrawal sizing

Severity: **Low**	Impact: **Low**	Likelihood: **Low**
Files: src/modules/lend-gateway/LendCapacityLib.sol	Status: Fixed	

Description: `LendCapacityLib.borrowCapacity` and `withdrawHeadroom` both clamp negative results to 0, discarding how far underwater a position already is. `_resupplyForCreditShortfall` uses the clamped `rawBorrowCapacity` to size collateral for only the incremental new borrow, never for a pre-existing shortfall, so the subsequent `gateway.borrow` reverts on Aave's own health check. `repayWithdrawable` adds the repay's marginal freed value (repayValue) on top of the clamped `rawWithdrawHeadroom`, over-crediting withdrawal headroom by the deficit amount when one exists, so the sized withdrawal reverts too. Both failures occur precisely when a safe is already in an unhealthy state — exactly when these paths (credit settlement, repay) are relied on to recover it.

Recommendations: Track and propagate the true (possibly negative) deficit alongside the clamped capacity/headroom values, or expose an explicit "current deficit" accessor.

Customer's response: Fixed in commit 098cb3e

Fix Review: Fixed

Informational Severity Issues

I-01: EtherFi operations dependent on Aave Hub liquidity

Files:

src/modules/lend-gateway/LendGateway.sol

Description:

This Hub-wide idle cash is shared across every Spoke and borrower. Consequently, a fully-funded debit spend reverts, withdrawals are blocked, and lend opt-out is bricked purely because an unrelated third party drew down the Hub's cash. There is no fallback as the module keeps no reserve buffer loose.

Recommendations:

Maintain a buffer for the sole purpose of ensuring card spends can operate successfully.

Customer Response:

Acknowledged

Fix Review:

Acknowledged

I-02: An Aave reserve pause blocks Debit spending even when all required funds are loose in the Safe

Files:

src/modules/lend-gateway/LendGateway.sol

Description:

`isSpendAsset` combines whether the asset is a card settlement token and whether the corresponding Aave reserve is paused. Both CashLens and Debit execution call this predicate before checking loose balances.

As a result, an Aave reserve pause rejects the transaction even when no Aave withdrawal is necessary. This forces an opted-out Safe into Debit mode but blocks its card settlement entirely during an Aave pause, despite the Safe no longer depending on Aave for that token.

Recommendations:

Separate static spend-token membership from Aave withdrawal availability. Source as much as possible from the loose balance and consult pause/liquidity state only when a supplied-balance withdrawal is actually required.

Customer Response:

Fixed in commit 636809c

Fix Review:

Fixed

I-03: Owner-signed borrow authorizations have no contract domain or expiry

Files:

src/modules/cash/CashLendLib.sol

Description:

The owner-quorum borrow digest verification completely omits both the verifying module address and an explicit expiry deadline. An unexecuted borrow authorization remains valid indefinitely as long as the Safe nonce remains unchanged, and lacks domain separation.

Recommendations:

Use EIP-712 with the CashModule proxy as the verifying contract and include an explicit `deadline` in the payload.

Customer Response:

Acknowledged - "*Expiry declined: the shared safe nonce is burned by any later quorum op, and a borrow signature binds an exact token and amount. Domain separation agreed in principle, pending scope — all seven quorum flows share the gap — and off-chain signer coordination.*"

Fix Review:

Acknowledged

I-04: Zero-factor collateral is incorrectly treated as unconditionally withdrawable

Files:

src/modules/lend-gateway/LendCapacityLib.sol

Description:

Zero-factor collateral returns zero weight and is assumed entirely withdrawable without health checks, but Aave validates health on withdrawal if the collateral flag is on, potentially reverting valid executions.

Unconditionally return the full supply only when collateral usage is disabled, else mirror live health validation limits.

Customer Response:

Acknowledged - *"A zero-factor withdrawal is health-neutral in Aave's accounting (the risk-premium loop is gated on `collateralFactor > 0`), so it only fails when the position is already below 1.00."*

Fix Review:

Acknowledged

I-05: Potential to bypass Lend operations and supply and withdraw from Aave directly if safes implement ERC1271

Files:

/lib/aave-v4/spoke/Spoke.sol

Description:

There exists a function `setUserPositionManagersWithSig`. Currently safe's do not implement ERC-1271 signatures so it is impossible for a safe to call it, but if you implement it in the future, safe's could bypass lend and withdraw directly.

Recommendations:

Do not implement ERC-1271 on the safe contract.

Customer Response:

Acknowledged - *"Agreed, and already an explicit invariant: the `LendGateway` header records that `EtherFiSafe` must never implement ERC-1271, precisely because it would arm `setUserPositionManagersWithSig`."*

Fix Review:

Acknowledged

I-06. The LiquidUSD repayment path can worsen Aave health while bypassing the configured floor

Description: The gateway branch repays USDC debt and then takes `LiquidUSD` collateral from the Safe as reimbursement. It deliberately omits the minimum-health-factor check under the assumption that replacing collateral with an equal debt reduction is always de-risking.

That assumption is not guaranteed. Removing `LiquidUSD` collateral applies `LiquidUSD's` collateral factor, while repaying USDC removes debt value without that factor. The `Cash` and Aave oracle prices can also diverge. Consequently, the Aave-weighted collateral removed can exceed the risk value freed by the USDC repayment, worsening the health factor and potentially dropping the position below the configured minimum health factor.

Recommendation: Do a before/after Aave health check to ensure position was actually improved

Customer's response: Acknowledged - "*Possible only in a narrow regime: health worsens when the weighted collateral removed per unit of debt repaid exceeds the current health factor, needing a high collateral factor plus adverse oracle divergence at a low health factor.*"

Fix Review: Acknowledged

I-07. Repayment sourcing ignores liquidity created by its own first repayment leg

Description: `repayWithdrawable` calculates withdrawal limits prior to the state changing loose-token repayment leg, ignoring the Hub liquidity that the first leg actually frees up. This could cause repayments to revert prematurely for insufficient balance.

Recommendation: Consider executing the `loose` repayment first and recompute withdrawal liquidity and headroom from the live post-repayment state.

Customer's response: Fixed in commit ea534817

Fix Review: Fixed

I-08. _requestWithdrawal accepts empty token lists and zero-amount entries

Description: _requestWithdrawal stores a WithdrawalRequest without validating that tokens is non-empty or that each amounts[i] != 0.

Recommendation: In _requestWithdrawal, revert if tokens.length == 0, and revert if any amounts[i] == 0 — matching the validation already enforced independently by each module that calls requestWithdrawalByModule.

Customer's response: Fixed in commit f4dec3e

Fix Review: Fixed

I-09. isDriver role grants unrestricted withdraw/borrow-to-arbitrary-address power over every gateway safe

Description: `LendGateway.onlyDriver` is the sole gate on withdraw/borrow, both of which accept a caller-supplied to. Any account granted `isDriver` can withdraw or borrow the full position of any registered safe to an address of its choosing, bypassing the owner-quorum and admin-signature checks that gate every other extraction path in the system. `setDriver` is admin-only, so this depends entirely on operational discipline: granting the role to an EOA, or any key usable without additional consent, is equivalent to giving that key custody of every migrated safe's Aave position.

Recommendation: `isDriver` to purpose-built contracts that internally scope and validate fund movement (as existing drivers do), never to an EOA or a freely-usable key.

Customer's response: Fixed - "Agreed, and already our practice: drivers are contracts only — the CashModule, the DebtManager, and the sandwiched modules — and no EOA is ever granted the role."

Fix Review: Fixed

I-10. processWithdrawal doesn't defer to the safe's current headroom needs before releasing reserved funds

Description: Once a gateway-safe withdrawal is requested, the pulled tokens sit loose for the entire delay, excluded from the auto-supply sweep so they're never resupplied as collateral. If the safe's health/headroom degrades during that delay, other flows (debit/credit spend, repay) treat this reserved balance as reclaimable — they call `cancelOldWithdrawal` and resupply it when they're short on headroom. `_processWithdrawal` has no equivalent check: it unconditionally transfers the reserved funds out regardless of the safe's health at finalize time, even in a case where canceling and resupplying would be needed to keep the position healthy or let a competing operation succeed.

Recommendation: Before finalizing a gateway-safe withdrawal, check the safe's current health/headroom. If the safe is unhealthy (or the withdrawal would push it there), prefer canceling and resupplying the reserved funds over releasing them, consistent with the priority already given to spend/repay's reclaim path, instead of unconditionally executing the transfer.

Customer's response: Acknowledged - "*The funds are pulled out of Aave at request time, so finalizing transfers non-collateral funds and leaves the health factor unchanged; the withdrawal cannot push the safe anywhere.*"

Fix Review: Acknowledged

I-11. getSafeCashData.maxBorrow clamps to debtUsd on an over-LTV position, hiding the breach

Description: `maxBorrow = account.availableBorrowsUsd + account.debtUsd`, and `availableBorrowsUsd` is clamped to 0 whenever debt reaches or exceeds the weighted max-borrow capacity. This makes maxBorrow equal totalBorrow exactly in that state, indistinguishable from a healthy position sitting precisely at its limit. SafeCashData carries no healthFactor or weighted-capacity field to recover the actual deficit, so risk tooling built on this view cannot detect or measure an over-LTV breach.

Recommendation: Expose the unclamped weighted max-borrow (or healthFactor) in SafeCashData so maxBorrow (or a companion field) can go below totalBorrow to signal and quantify an over-LTV position, rather than floor at debtUsd.

Customer's response: Fixed in commit a63469a

Fix Review: Fixed

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.